1. Data Structures

These are the building blocks of problem-solving. You must know how to implement and use them efficiently.

Arrays and Strings:

Key problems: Two-pointer technique, sliding window, Kadane's algorithm, string manipulation, and pattern matching.

Linked Lists:

Key problems: Reversal, cycle detection, merging, and intersection of linked lists.

Stacks and Queues:

Key problems: Implementing stacks/queues, using them for problems like valid parentheses, next greater element, and sliding window maximum.

Trees:

Key problems: Binary tree traversals (inorder, preorder, postorder), BST operations, LCA (Lowest Common Ancestor), and tree height/diameter.

Graphs:

Key problems: BFS, DFS, topological sorting, shortest path algorithms (Dijkstra, Bellman-Ford), and cycle detection.

Heaps:

Key problems: Implementing min-heap and max-heap, finding kth largest/smallest elements, and merging k sorted lists.

Hash Maps/Sets:

Key problems: Frequency counting, two-sum, subarray sum, and duplicate detection.

2. Algorithms

Mastering algorithms is crucial for solving problems efficiently.

Sorting and Searching:

Key algorithms: Merge sort, quick sort, binary search.

Key problems: Searching in rotated sorted arrays, finding first/last occurrence in sorted arrays.

Dynamic Programming (DP):

Key problems: Knapsack, longest common subsequence (LCS), longest increasing subsequence (LIS), matrix chain multiplication, and coin change.

Greedy Algorithms:

Key problems: Activity selection, fractional knapsack, and job scheduling.

Backtracking:

Key problems: N-Queens, Sudoku solver, and generating permutations/combinations.

Divide and Conquer:

Key problems: Merge sort, quick sort, and finding maximum subarray sum.

Sliding Window:

Key problems: Maximum sum subarray of size k, longest substring with k distinct characters.

Two-pointer Technique:

Key problems: Pair with target sum, removing duplicates from a sorted array.

3. Problem-Solving Patterns

FAANG interviews often test your ability to recognize and apply common problem-solving patterns.

Frequency Counting:

Problems involving counting elements or finding duplicates.

Prefix Sum:

Problems involving subarray sums or cumulative calculations.

Bit Manipulation:

Key problems: Counting set bits, finding unique numbers, and bitwise operations.

Union-Find (Disjoint Set):

Key problems: Detecting cycles in undirected graphs and connected components.

Trie:

Key problems: Autocomplete, word search, and prefix matching.

4. System Design (For Experienced Candidates)

While not part of the coding round, system design is often asked for experienced candidates or in later rounds.

Key topics: Design URL shortener, design a cache, design Twitter/Instagram, and scalable database design.

5. Behavioral and Problem-Solving Skills

FAANG interviews also assess your ability to think critically and communicate effectively.

Approach to Problem-Solving:

Always start by clarifying the problem, discussing edge cases, and explaining your thought process before coding.

Optimization:

Be prepared to optimize your solution in terms of time and space complexity.

Communication:

Clearly explain your approach and code to the interviewer.